That’s when I pivoted to encryption.

Deletion is a dream for the surveillant and a nightmare for the surveilled, but encryption is, or should be, a reality for all. It is the only true protection against surveillance. If the whole of your storage drive is encrypted to begin with, your adversaries can’t rummage through it for deleted files, or for anything else—unless they have the encryption key. If all the emails in your inbox are encrypted, Google can’t read them to profile you—unless they have the encryption key. If all your communications that pass through hostile Australian or British or American or Chinese or Russian networks are encrypted, spies can’t read them—unless they have the encryption key. This is the ordering principle of encryption: all power to the key holder.

Encryption works, I explained, by way of algorithms. An encryption algorithm sounds intimidating, and certainly looks intimidating when written out, but its concept is quite elementary. It’s a mathematical method of reversibly transforming information—such as your emails, phone calls, photos, videos, and files—in such a way that it becomes incomprehensible to anyone who doesn’t have a copy of the encryption key. You can think of a modern encryption algorithm as a magic wand that you can wave over a document to change each letter into a language that only you and those you trust can read, and the encryption key as the unique magic words that complete the incantation and put the wand to work. It doesn’t matter how many people know that you used the wand, so long as you can keep your personal magic words from the people you don’t trust.

Encryption algorithms are basically just sets of math problems designed to be incredibly difficult even for computers to solve. The encryption key is the one clue that allows a computer to solve the particular set of math problems being used. You push your readable data, called plaintext, into one end of an encryption algorithm, and incomprehensible gibberish, called ciphertext, comes out the other end. When somebody wants to read the ciphertext, they feed it back into the algorithm along with—crucially—the correct key, and out comes the plaintext again. While different algorithms provide different degrees of protection, the security of an encryption key is often based on its length, which indicates the level of difficulty involved in solving a specific algorithm’s underlying math problem. In algorithms that correlate longer keys with better security, the improvement is exponential. If we presume that an attacker takes one day to crack a 64-bit key—which scrambles your data in one of 2^{64} possible ways (18,446,744,073,709,551,616 unique permutations)—then it would take